LD8-S2 Canonical Target-Owned Direct Realization Specification

Exact finite-stencil accumulation, bounded vectorized workspaces, immediate packed output, and migration equivalence

mdstats development specification

2026-07-21

Contents

1 LD8-S2 - Canonical target-owned direct realization

Package target: `mdstats 0.19.56a0`
Status: implemented
Primary module:

`mdstats.plotting.density_block_direct`

1.1 Purpose

LD8-S2 realizes scientific density values directly from the exact LD8-S1 support atlas. It preserves the existing periodic CIC-plus-normalized-Gaussian estimator while replacing LD7's source-group partial-field construction with one globally aggregated source field and one deterministic owner for each target block.

S2 is the canonical migration oracle for later hybrid execution. It is not yet the default full-production executor. LD7 remains the production fallback until LD8-S3 and the LD8-S4 performance gate pass.

1.2 Scientific estimator

Let $m_{\mathrm{CIC}}(\mathbf{n})$ be the globally aggregated periodic CIC source and let $g(\delta)$ be the exact normalized finite Gaussian stencil retained at

$$\varepsilon_{\text{tail}} = 10^{-8}.$$

The target node mass is

$$M(\mathbf{t}) = \sum_{\delta \in \mathcal{K}_\varepsilon} m_{\text{CIC}}((\mathbf{t} - \delta) \bmod \mathbf{N})\, g(\delta).$$

The scientific density is

$$\rho(\mathbf{t}) = \frac{M(\mathbf{t})}{\Delta V}, \qquad \Delta V = \frac{|\det H|}{N_x N_y N_z}.$$

The support atlas is exact, so every stored target node has at least one strictly positive contribution and every node outside the atlas is an exact implicit zero.

1.3 Public records

```
@dataclass(frozen=True, slots=True)
class DensityDirectRealizationLimits:
    max_target_blocks: int
    max_target_nodes: int
    max_candidate_pairs: int
    max_exact_contributions: int
    max_pair_chunk_size: int
    max_transient_bytes: int
    max_retained_bytes: int

@dataclass(frozen=True, slots=True)
class DensityDirectRealizationPlan:
    source_field_identity: str
    routing_identity: str
    atlas_identity: str
    stencil_identity: str
    target_block_count: int
    target_support_node_count: int
    source_target_edge_count: int
    exact_contribution_count: int
    conservative_candidate_pair_count: int
    pair_chunk_size: int
    source_coordinate_bytes: int
    reverse_csr_bytes: int
    peak_pair_workspace_bytes: int
    accumulator_bytes: int
    packed_field_bytes_upper: int
    predicted_peak_bytes: int
```

The plan is immutable, JSON serializable, and tied to exact content identities. It is not reusable for a different source field or atlas.

1.4 API

```
def plan_target_owned_direct_realization(
    source_field: PeriodicPackedCICSourceField3D,
    stencil: PeriodicGaussianStencilSupport,
    routing: PeriodicKernelBlockRouting,
    atlas: DensitySupportAtlas,
    *,
```

```
    limits: DensityDirectRealizationLimits | None = None,
) -> DensityDirectRealizationPlan


def realize_density_target_owned_direct(
    source_field: PeriodicPackedCICSourceField3D,
    stencil: PeriodicGaussianStencilSupport,
    routing: PeriodicKernelBlockRouting,
    atlas: DensitySupportAtlas,
    *,
    field_key: str,
    label: str,
    physical_units: str,
    broadening_metric: str,
    limits: DensityDirectRealizationLimits | None = None,
    approved_plan: DensityDirectRealizationPlan | None = None,
) -> PeriodicPackedBlockScalarField3D
```

1.5 Input identity and compatibility constraints

Before planning or realization:

1. source, stencil, routing, and atlas must share the same logical grid;
2. source, routing, and atlas must share the same storage-block shape;
3. `source_field.content_identity == atlas.source_field_identity`;
4. `routing.cache_identity == atlas.routing_identity`;
5. `stencil_content_identity(stencil) == routing.stencil_identity`;
6. the approved plan, when supplied, must match all four identities;
7. labels, field keys, units, and broadening metric must be nonempty strings.

A mismatch raises `GraphAdapterError` before numerical allocation.

1.6 Transactional planning

Planning builds only bounded coordinate summaries and source/target CSR routing. It does not allocate a floating target field.

For each source-target block edge, the planner computes a conservative set of stencil offsets whose translated occupied-source bounding box may intersect the target block. This block-level test may overestimate numerical pairs but may never omit an exact contribution.

The exact contribution count is

$$P_{\text{exact}} = N_{\text{source nodes}} |\mathcal{K}_\varepsilon|.$$

The conservative vectorized candidate count is

$$P_{\text{candidate}} = \sum_{(s,t)\in E} N_s K_{s\to t},$$

where $K_{s\to t}$ is the number of stencil offsets whose translated source-block bounding interval can intersect target block t.

Planning fails before realization if any declared target, contribution, candidate-pair, retained-byte, or transient-byte limit is exceeded.

1.7 Canonical execution order

The normative deterministic order is:

```
target block: canonical C-order
  source block: canonical source-row order
    stencil offset: canonical stencil active-index order
      source node: canonical local C-order
```

Python may schedule target/source blocks and vectorized chunks. It must not iterate in Python over individual source-node/stencil pairs.

1.8 Bounded vectorized kernel

For one source-target edge:

1. select only stencil offsets whose translated occupied-source bounding interval may intersect the target block;
2. choose an offset chunk satisfying `offset_count * source_node_count <= max_pair_chunk_size`; when one occupied source block itself exceeds the limit, use one stencil offset and split source nodes in canonical local C-order;
3. form exact periodic target coordinates for the chunk;
4. retain coordinates inside the owned target block;
5. accumulate accepted contributions with a compiled NumPy reduction into one reusable dense $B_x B_y B_z$ target accumulator;
6. release all chunk arrays before processing the next chunk.

No global target-coordinate array and no complete source-node by stencil-offset pair array may be allocated.

1.9 Target ownership and immediate packing

Each target block has one owner and one reusable dense accumulator. After all contributing source blocks are processed:

1. the positive local-node set must equal the atlas support bitset exactly;
2. positive masses are written in local C-order into one preallocated packed vector whose offsets are resolved from atlas bit counts before realization;
3. the dense accumulator is cleared and reused;
4. no dense completed target block remains resident.

The target occupancy bitsets are inherited from the exact atlas. Packed values remain unnormalized masses until all target blocks are complete. The implementation must not retain a list of completed block arrays or concatenate a second full packed vector. Conversion from mass to density is in place. The immutable output constructor may hold one validation copy transiently; the plan therefore counts two retained-output vectors in its conservative peak estimate.

1.10 Final normalization

Let

$$M_{\mathrm{raw}} = \sum_i M_i.$$

Apply one common factor

$$\alpha = \frac{M_{\mathrm{target}}}{M_{\mathrm{raw}}}.$$

A final deterministic floating residual is applied to the largest positive mass, which provides the greatest positivity margin. The corrected masses must remain finite and strictly positive. Densities are then obtained by division by ΔV.

Block extrema are computed during final packing. A block minimum is zero whenever the positive support does not fill every valid node of that block.

1.11 Output metadata

The packed field records at least:

```
reference_path = ld8_s2_canonical_target_owned_direct
production_backend = false
source_field_identity
routing_identity
atlas_identity
stencil_identity
exact_contribution_count
conservative_candidate_pair_count
accepted_contribution_count
vectorized_chunk_count
peak_chunk_pair_count
source_coordinate_bytes
reverse_csr_bytes
peak_pair_workspace_bytes
packed_field_bytes
raw_measure_before_final_normalization
final_normalization_factor
mass_correction_index
final_measure
complete_fine_pair_array_allocated = false
global_target_coordinate_array_allocated = false
completed_dense_target_blocks_retained = false
```

1.12 Failure semantics

`GraphAdapterError` incompatible identities, support mismatch, invalid terminal slot, nonpositive output, normalization failure, or realized counts inconsistent with the approved plan.
`GraphComplexityError` target, pair, transient-workspace, or retained-output limit exceeded.
`GraphStyleError` invalid limits, empty labels, units, or broadening metric.

No failure may relax the kernel cutoff, broaden the Gaussian, coarsen the grid, omit support nodes, or fall back silently.

1.13 Required focused tests

1. exact agreement with explicit periodic convolution on small grids;
2. relative L^1 agreement with LD1-A and LD7 no weaker than 5×10^{-12};
3. periodic face, edge, and corner crossings;
4. partial terminal blocks on one, two, and three axes;
5. overlapping source contributions;
6. exact support-bitset equality after realization;
7. exact target measure and units;
8. deterministic repeated output and content identity;
9. translation covariance under periodic integer shifts;
10. identity mismatch rejection before allocation;
11. candidate-pair and transient-byte preflight rejection;
12. no complete fine-pair array or global target-coordinate array;
13. packed output smaller than fixed dense active-block storage on localized fixtures;
14. pair limits smaller than one occupied source block are honored exactly.

1.14 Recorded implementation evidence

The bounded production-stencil benchmark used a 96^3 logical grid, a 17 angstrom orthogonal cell, $\sigma = 2h$, and the retained 10^{-8} Gaussian-tail cutoff. The exact stencil contained 8,409 offsets. Across 64, 128, and 512 distributed source nodes:

Source nodes	Exact contributions	Candidate pairs	Target nodes	S2 time	LD1-A time	Relative L^1	S2 packed bytes
64	538,176	676,756	407,999	0.502 s	0.328 s	1.47×10^{-18}	3,376,340
128	1,076,352	1,938,781	609,687	0.881 s	0.442 s	2.13×10^{-16}	4,995,872
512	4,305,408	14,235,800	880,724	2.526 s	0.801 s	4.10×10^{-17}	7,164,168

The S2 packed representation retained approximately half the bytes of the LD1-A flat-index/value representation on these fixtures. The current NumPy target-owned oracle was 1.5–3.2 times slower than LD1-A. This is acceptable for the canonical migration oracle but explicitly does not authorize production dispatch. LD8-S3 remains responsible for compiled/tiled direct and overlap-add FFT acceleration plus crossover selection.

1.15 Deferred work

LD8-S2 does not:

- replace the production LD7 dispatch;
- implement tiled overlap-add FFT;
- auto-select direct versus FFT execution;
- implement weighted multi-HDR selection;
- optimize contour extraction or browser rendering.

Those changes belong to LD8-S3, LD8-S4, and LD9.

1.16 Attribution

Periodic CIC assignment follows Hockney and Eastwood, *Computer Simulation Using Particles* (1988), as already cited by the source-deposition specification. The finite normalized Gaussian stencil is governed by the existing kernel specification. Target ownership, source-target bounding-interval pruning, immediate packed realization, and the identity-bound planning contract are mdstats-specific designs.